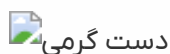


دست گرمی

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت

با دست گرمی شروع می‌کنیم.



دست گرمی

برنامه‌ی زیر را طوری تکمیل کنید که جای عناصر را به کمک Call by Reference با هم عوض کند.

```
1  #include <stdio.h>
2
3  void swapNumbers(int *x, int *y, int *z);
4
5  int main() {
6      int e1, e2, e3;
7
8      scanf("%d %d %d", &e1, &e2, &e3);
9
10     // TODO: Call the function to swap the values of the three element
11
12     printf("%d %d %d", e1, e2, e3);
13
14     return 0;
15 }
16
17 // Function definition to swap the values of three integer pointers
18 void swapNumbers(int *x, int *y, int *z) {
19     // TODO: code here.
20 }
```

مثال

ورودی نمونه ۱

3 6 9

خروجی نمونه ۱

9 3 6

دودویی

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت

ابتدا توضیحات مسئله را از این لینک ببینید:

[کلیک کنید](#)

الگوریتم جست‌وجوی دودویی بین اعداد 1 تا n را در نظر بگیرید: شما یک آرایه‌ی مرتب از این اعداد دارید و در ابتدا میانه‌ی آن را در نظر می‌گیرید. سپس عددی که به دنبالش هستید را با آن مقایسه کرده و اگر بزرگ‌تر بود سراغ نیمه‌ی بزرگ‌تر و در غیر این صورت سراغ نیمه‌ی کوچک‌تر رفته و همین فرآیند را به صورت بازگشتی تکرار می‌کنید.

برای مثال برای جست‌وجو میان اعداد 1 تا 7، درخت زیر تمام مسیرهای ممکن را نشان می‌دهد:



در این سوال عدد n به شما داده می‌شود (حداکثر ۱۰۲۳) و شما باید درخت بالا را برایش بسازید. گره‌های این درخت باید از استراکت زیر تشکیل شده باشند:

```
1 | typedef struct Node {  
2 |     int value;  
3 |     struct Node *left;  
4 |     struct Node *right;  
5 | } Node;
```

برای مثال اگر اعداد ۱ تا ۷ را در نظر بگیرید، مقدار ریشه ۴ است و فرزند راست آن گرهی است که مقدار ۶ دارد و فرزند چپ آن گرهی با مقدار ۲ است.

در این مسئله اگر میانه اعشاری بود، آن را رو به پایین گرد کنید.

نکات مهم

۱. شما باید تابعی پیاده‌سازی کنید که دقیقاً به شکل زیر باشد که در آن `n` ورودی مسئله است. همچنین `answer` نیز خروجی نهایی کد شماست که آدرس ریشه‌ی درخت می‌باشد.

```
1 Node* build_structure(int n ) {  
2     // your code  
3     return answer;  
4 }
```

۲. نام ورودی تابع را تغییر ندهید.

۲. در کدی که ارسال می‌کنید نباید تابع `main` وجود داشته باشد. ما خروجی تابع `build_structure` را داوری می‌کنیم و برنامه‌ی شما نباید چیزی چاپ کند. البته پیش از ارسال کدتان حتماً آن را تست کنید.

۳. پیشنهاد می‌شود برای تمیزی کد، توابع کمکی بنویسید و از آنها استفاده کنید.

۴. قبل از ارسال، فرمت کد خود را از `c` به `cpp` تغییر دهید. (دقت کنید این کار تنها برای داوری سوال مورد نیاز است و شما مجاز به استفاده از قابلیت‌های زبان `C++` نیستید.)

مثال

ورودی نمونه ۱

3

خروجی نمونه ۱

```
pointer to this node:  
node.value=2  
node.left=pointer to node 1  
node.right=pointer to node 3
```

عبارت بالا را صرفاً برای توضیح نوشتیم و شما نباید چیزی چاپ کنید.

گربه‌های فضایی

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت

چگونه گربه‌ها روی پاهایشان فرود می‌آیند؟



توضیح تصویر

در این سوال یک ماتریس مربعی به شما در ورودی داده می‌شود. سپس یک سری عملیات‌هایی که در ادامه توضیح داده می‌شوند روی ماتریس اعمال می‌شوند.

ورودی

ابتدا در یک خط عدد n در ورودی داده می‌شود. سپس در خط بعد به تعداد n^2 عدد داده می‌شود که این اعداد، مقادیر موجود در درایه های ماتریس را نشان می‌دهند. ترتیب این اعداد به این صورت است: اگر ماتریس ورودی را داشته باشیم و از خانه‌ی گوشه چپ بالا به صورت مارپیچ شروع به خواندن اعداد درون ماتریس کنیم و اعدادی که می‌خوانیم را در یک سطر به ترتیب بنویسیم، سطر به دست آمده همان سطری است که در ورودی داده می‌شود. به طور مثال اگر ورودی به صورت زیر باشد:

3
1 2 3 4 5 6 7 8 9

ماتریسی که در واقع ماتریس ورودی سوال ماست به شکل زیر می‌باشد :

1 2 3
8 9 4
7 6 5



نحوه خواندن ماتریس از روی ورودی

زیرا همانطور که در تصویر بالا مشاهده میکنید اگر اعداد ماتریس مذکور را به صورت مارپیچی بخوانیم دنباله ی ورودی (1 2 3 4 5 6 7 8 9) به دست می‌آید.

عملیات ها

ترنسپوز کردن

```
transpose
```

با آمدن این دستور شما ماتریس سوال را با ترانهاده اش عوض می کنید.

▼ تعریف ترانهاده

ترانهاده یعنی سطر و ستون را جابه جا کنیم. به عبارت دیگر باید هنگام نوشتن ترانهاده هر ماتریسی سطرهای ماتریس را به شکل ستون نوشت و ستونهای ماتریس را به شکل سطر؛ به طور مثال:

 توضیح تصویر

دوران ماتریس

```
rotate <clockwise|counter clockwise>
```

با آمدن این دستور درایه ها ماتریس را 90 درجه دوران می دهیم. اگر بخش دوم دستور clockwise بود دوران ساعتگرد و اگر counter clockwise بود دوران پادساعتگرد خواهد بود.

به طور مثال ماتریس

```
1 2 3
4 5 6
7 8 9
```

با دوران ساعتگرد به:

```
7 4 1
8 5 2
9 6 3
```

و با دوران پادساعتگرد به:

```
3 6 9
2 5 8
1 4 7
```

تبدیل می شود.

جا به جا کردن دو سطر

```
swap rows a and b
```

با آمدن این دستور دو سطر شماره a و b ماتریس با یکدیگر جا به جا می شوند.

به طور مثال اگر دستور زیر آمده باشد:

```
swap rows 1 and 3
```

و ماتریس ما ماتریس زیر باشد:

```
1 2 3
4 5 6
7 8 9
```

ماتریس جدید خواهد شد:

```
7 8 9
4 5 6
1 2 3
```

جا به جا کردن دو ستون

```
swap columns a and b
```

با آمدن این دستور دو ستون شماره a و b ماتریس با یکدیگر جا به جا می شوند.

به طور مثال اگر دستور زیر آمده باشد:

```
swap columns 1 and 3
```

و ماتریس ما ماتریس زیر باشد:

```
1 2 3
4 5 6
7 8 9
```

ماتریس جدید خواهد شد:

```
3 2 1
6 5 4
9 8 7
```

در دو دستور بالا، اگر دو عددی که به جای a یا b آمده اند شماره سطر یا ستون نا معتبر باشند ، عملیاتی انجام ندهید و خط زیر را چاپ کنید:

```
invalid numbers for swapping.
```

چاپ نقاط زینی

```
saddle
```

با آمدن این دستور تمام نقاط زینی به فرمتی که در مثال زیر آمده است چاپ می شوند. نقطه زینی در یک ماتریس **عنصری است که در سطر خود کمینه (کوچکتر یا مساوی بقیه) و در ستون خود بیشینه (بزرگتر یا مساوی بقیه) باشد**. سطر ها را از بالا به پایین و ستون ها را از چپ به راست شماره گذاری می کنیم. سطر ها و ستون ها را با اعداد 1 تا n شماره گذاری می کنیم. هر خانه را به صورت (row, column) نشان می دهیم که row عدد سطر آن و column عدد ستون آن می باشد.

به طور مثال اگر این دستور آمده باشد و ماتریس ما ماتریس زیر باشد:


```
3 4 5
2 6 7
3 5 6
```

خروجی خواهد شد:

```
(1, 1)(3, 1)
```

اگر نقطه زینی‌ای نداشتیم یک خط خالی چاپ کنید. ترتیب چاپ نقاط نیز به این صورت است که ابتدا نقاط زینی مربوط به سطر اول از چپ به راست چاپ می‌شود. سپس به همین ترتیب نقاط زینی سطر دوم چاپ می‌گردد. به همین ترتیب برای سطرهای بعدی.

ورودی

ابتدا عدد طبیعی n در ورودی می‌آید. سپس در خط بعد n^2 عدد طبیعی پشت سر هم (با یک اسپیس جدا می‌شوند) می‌آیند که داده‌های ماتریس را به صورت مارپیچی نمایش می‌دهند. در خط‌های بعدی نیز دستورات به ترتیب در ورودی نوشته می‌شوند تا در نهایت به `end` برسیم و برنامه به پایان برسد.

خروجی

در طول برنامه در بخش‌های مورد نیاز در حین اجرای برنامه، خروجی‌هایی که در بعضی دستورات ذکر شده است را چاپ کنید. در انتهای برنامه نیز ماتریس نهایی را در خروجی چاپ کنید (هر سطر ماتریس در یک خط و بین درایه‌های یک خط یک عدد اسپیس بگذارید).

ورودی نمونه ۱:

```
4
9 16 10 1 6 7 1 8 20 10 19 17 8 17 2 14
swap columns 9 and 2
rotate counter clockwise
swap rows 3 and 1
transpose
rotate clockwise
end
```

خروجی نمونه ۱:

```
invalid numbers for swapping.  
20 14 8 16  
8 2 17 10  
1 7 6 1  
10 19 17 9
```

ورودی نمونه ۲:

```
2  
2 16 4 11  
rotate counter clockwise  
transpose  
swap columns 2 and 1  
rotate clockwise  
swap rows 1 and 2  
saddle  
end
```

خروجی نمونه ۲:

```
4 16  
11 2
```

ورودی نمونه ۳:

```
11  
11 98 94 99 94 100 94 98 100 100 98 24 29 8 83 99 86 8 55 17 83 29 53 32 56  
saddle  
transpose  
swap columns 13 and 3  
swap rows 3 and 3  
rotate clockwise  
swap rows 9 and -1  
end
```

خروجی نمونه ۳:

```
(5, 1)(7, 1)
invalid numbers for swapping.
invalid numbers for swapping.
98 100 100 98 94 100 94 99 94 98 11
24 2 92 44 58 93 57 45 37 85 11
29 23 81 34 6 40 28 20 53 2 14
8 6 54 15 45 47 18 93 56 10 14
83 19 82 54 65 95 59 73 94 55 14
99 29 34 90 48 100 17 35 65 30 11
86 96 37 87 67 63 68 46 69 64 14
8 7 65 33 38 57 44 85 1 45 8
55 17 59 7 25 40 46 94 51 16 1
17 28 11 43 48 75 15 36 62 38 1
83 29 53 32 56 50 68 27 41 78 3
```

سامانه ترافیکی

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت

اگر یک تکه تست مربایی را به یک کفش بچسبانیم و به هوا پرتاب کنیم، از چه سمتی فرود می‌آید؟



در این سوال شما یک سامانه ترافیکی را شبیه سازی می کنید. تصور کنید یک جاده ی یک طرفه دارید که دو لاین ماشین دارد. با توجه به دستورات داده شده، تغییرات لازم را روی لاین ها و ماشین ها اعمال می کنید. توجه کنید که هر ماشین یک پلاک یکتا دارد. همچنین هر ماشین یک رنگ نیز دارد. فرض کنید منظورمان از لاین شماره صفر، لاین سمت چپ و لاین شماره یک، لاین سمت راست است. توجه کنید در هر بخش خطا ها به ترتیب گفته شود باید چاپ شود.

لیست عملیات ها

اضافه کردن ماشین

```
add <lane> <k> <plate> <color>
```

در این دستور شما ماشینی با رنگ `color` و پلاک `plate` را می سازید و در لاین شماره `lane` در جایگاه `k` ام آن لاین قرار می دهید. تضمین می شود که `lane` برابر صفر یا یک می باشد.

اگر پلاک داده شده تکراری باشد (ماشینی در جاده با آن پلاک وجود داشته باشد) عملیاتی انجام نمی دهید و عبارت زیر را چاپ می کنید:

```
this plate was already given
```

اگر k کوچکتر از یک بود یا بزرگتر از تعداد ماشین های آن لاین + 1 بود، عملیاتی انجام نمی دهید و در خطی جدا عبارت زیر را چاپ می کنید.

an invalid number is given

در واقع بعد از این عملیات ماشین جدید، در جایگاه k ام قرار می گیرد. لذا اگر $k=1$ در لاین مورد نظر، ماشین جدید اولین ماشین لاین می شود.

حذف ماشین

delete

در این دستور شما باید تمام ماشین هایی که رنگ آنها black است را از هر دو لاین به طور کلی حذف کنید.

عوض کردن لاین

change <plate>

در این دستور ماشینی که پلاک plate (که در دستور گفته شده) را دارد لاینش را عوض می کند. یعنی اگر لاین صفر است به لاین یک می رود و بالعکس.

توجه کنید که اگر ماشین در لاین خودش در جایگاه k ام قرار داشته باشد در لاین جدیدش نیز در جایگاه k ام قرار می گیرد.

اگر تعداد ماشین ها در لاین جدید کمتر از k بوده باشد، ماشین پشت سر آخرین ماشین لاین جدیدش قرار می گیرد. (آخرین ماشین در لاین جدید می شود).

اگر ماشینی با پلاک plate وجود نداشت، عملیاتی انجام ندهید و عبارت زیر را

Car not found

در خطی جداگانه چاپ کنید.

سبق

```
overtake <plate> <number>
```

در این دستور ماشینی که پلاک `plate` (که در دستور گفته شده) را دارد از `number` تا ماشین جلویی اش سبقت می گیرد. اگر `number` کوچکتر از یک بود عملیاتی انجام نمی دهید و در خطی جدا عبارت زیر را چاپ می کنید.

```
an invalid number is given
```

اگر ماشینی با پلاک `plate` وجود نداشت، عملیاتی انجام ندهید و عبارت زیر را

```
Car not found
```

در خطی جداگانه چاپ کنید.

اگر تعداد ماشین های جلویی از `number` کمتر بوده باشد، ماشین جلوی اولین ماشین لاین خودش قرار می گیرد. (جلوترین ماشین در لاینش می شود).

ورودی

در ورودی تعدادی از دستورات بالا داده می شود. هر گاه عبارت

```
EXIT
```

را در ورودی دریافت کردید برنامه تمام می شود و باید اطلاعات جاده را چاپ شود.

خروجی

در حین اجرای برنامه، خروجی هایی که در بعضی دستورات ذکر شده است را چاپ کنید. اطلاعات جاده را به فرمت زیر چاپ کنید:

```
lane 0:  
plate1 (color1)
```

```
plate2 (color2)
plate3 (color3)
*****
lane 1:
plate1 (color1)
plate2 (color2)
```

که در هر قسمت اطلاعات ماشین های هر لاین چاپ شده است.(توجه کنید در این مثال خط صفر، سه ماشین و خط یک، دو ماشین دارد.)

ورودی نمونه ۱:

```
add 0 1 ABC123 red
add 0 3 GHJ876 blue
add 1 1 XYZ789 blue
add 0 2 DEF456 black
overtake ABC123 1
change XYZ789
delete
EXIT
```

خروجی نمونه ۱:

```
an invalid number is given
lane 0:
XYZ789 (blue)
ABC123 (red)
*****
lane 1:
```

ورودی نمونه ۲:

```
add 0 1 AAA111 green
add 0 2 AAA111 black
add 0 2 BBB222 black
```

```
add 1 1 CCC333 blue
overtake CCC333 2
change AAA111
delete
overtake AAA111 1
EXIT
```

خروجی نمونه ۲:

```
this plate was already given
lane 0:
*****
lane 1:
AAA111 (green)
CCC333 (blue)
```

ورودی نمونه ۳:

```
add 0 1 LMY640 black
add 1 1 JAX738 green
add 1 1 MVZ342 green
add 1 1 AYY443 red
add 1 5 YAF316 blue
delete
add 1 4 DLF283 black
add 1 1 THS933 red
add 1 1 DYZ963 blue
delete
add 0 3 RCO424 black
add 1 1 IOB200 green
add 1 1 OAN201 green
add 0 1 YLP802 red
add 1 1 VFC756 red
change VFC756
add 0 1 ZDC218 blue
add 1 1 QRJ312 green
change AOD659
add 0 1 AJR450 red
add 1 1 QTQ585 black
```



```
change VFC756
change DIZ314
add 1 1 AYU232 green
add 0 3 RPD202 blue
add 0 1 AOD659 green
add 0 1 MEK893 blue
add 1 3 NSD642 black
change QRJ312
change MVZ342
change FIJ313
overtake MVZ342 1
change OAN201
change BAS631
add 0 1 DIZ314 black
add 1 1 RAD504 green
add 0 1 MCV175 red
overtake QTQ585 2
add 1 1 UQF390 red
add 0 1 BAS631 red
add 1 1 EVQ887 blue
add 0 2 KIX638 green
add 0 1 FIJ313 red
EXIT
```

خروجی نمونه ۳:

```
an invalid number is given
an invalid number is given
Car not found
Car not found
Car not found
Car not found
lane 0:
FIJ313 (red)
BAS631 (red)
KIX638 (green)
MCV175 (red)
DIZ314 (black)
MEK893 (blue)
AOD659 (green)
AJR450 (red)
```

QRJ312 (green)

OAN201 (green)

ZDC218 (blue)

RPD202 (blue)

MVZ342 (green)

YLP802 (red)

lane 1:

EVQ887 (blue)

UQF390 (red)

QTQ585 (black)

RAD504 (green)

AYU232 (green)

NSD642 (black)

VFC756 (red)

IOB200 (green)

DYZ963 (blue)

THS933 (red)

AYY443 (red)

JAX738 (green)

حیوانات شیء‌گرایانه

vs lbh frf guvf qbag gryy nalbar naq fraq n qhpx rzbwv va gur tebhcpung gb pbashfr rirelbar



هدف تمرین

شما باید توانایی خواندن تگه کدهای نوشته شده توسط همکارانتان و افزودن به آنها را داشته باشید. در این تمرین به شما یک تگه کد داده می‌شود و شما باید آن را تکمیل کنید. در این مسیر، یک آشنایی اولیه با مفاهیم برنامه‌نویسی شیء‌گرا نیز پیدا خواهید کرد.

در این کد، ما با استفاده از `enum`، `struct` و `pointer` ها، مفاهیم پایه‌ای OOP را در قالب یک زیست‌بوم از حیوانات شبیه‌سازی می‌کنیم.

اگر نمی‌دانید `enum` در زبان C چیست: فرض کنید یک متغیر دارید که می‌خواهید فصل سال را در آن بریزید. برای تمیز بودن کد، به شکل زیر عمل می‌کنیم:

```
1 enum season{
2     SPRING, SUMMER, AUTUMN, WINTER
3 };
4
5 int main() {
6     enum season s = WINTER;
7     printf("%d\n", s);
8     return 0;
9 }
```

(درواقع به صورت `int` ذخیره می‌شوند ولی کد را خوانا می‌کنند.)

ساختار کلی برنامه

همانند دنیای واقعی که هر حیوان هم یک دسته‌ی کلی دارد (مثلاً "پستاندار") و هم یک نوع خاص (مثلاً "سگ")، در کد ما نیز همین رابطه‌ی سلسله‌مراتبی ([Hierarchy](#)) بین ساختارها پیاده‌سازی شده است.

میتوانید تمپلیت این سوال که شامل ساختارها و توابع می‌باشد را از [این لینک](#) دانلود کنید.

در این تمرین، سه سطح اصلی از ساختارها (structها) وجود دارد که ارتباطی مشابه با کلاس‌ها و زیرکلاس‌های زبان‌های شی‌اگرا دارند:

1. Animal

نماینگر هر حیوان در زیست‌بوم است.

هر حیوان ویژگی‌های عمومی دارد مانند:

- `name` → نام حیوان
- `age` → سن حیوان
- `type` → اطلاعات مربوط به نوع و دسته‌ی آن (در قالب یک ساختار دیگر)
- چند تابع (از طریق function pointer) مانند `speak` یا `printInfo`

به زبان ساده، `Animal` پایه و ریشه‌ی تمام حیوانات است.

2. Category

هر حیوان متعلق به یک دسته (Category) است.

این دسته با استفاده از `enum` تعریف می‌شود تا فقط چند مقدار مشخص داشته باشد. در کنار این `enum`، ساختار `AnimalType` وجود دارد که نوع کلی حیوان و جزئیات مرتبط با آن را نگه می‌دارد:

```
1 | typedef struct AnimalType {
2 |     enum { MAMMAL, BIRD, REPTILE, FISH } category;
3 |     void* specificCategory; // اشاره‌گر به ساختار نوع خاص (Mammal, Bird, ...
4 | } AnimalType;
```

به کمک `void*`، می‌توانیم هر نوع دسته‌ای را درون `AnimalType` نگهداری کنیم بدون اینکه نوع دقیقش از قبل مشخص باشد — این همان مفهومی است که در OOP به عنوان **Polymorphism** می‌شناسیم. (نیازی به دونستن مفهومش نیست و فقط برای اطلاعات بیشتر آورده شده)

3. Specific (نوع خاص)

هر دسته از حیوانات (مثل Mammal , Bird , Reptile , Fish) می‌تواند چند حیوان خاص داشته باشد.
به عنوان مثال:

- Mammal → شامل Cat و Dog
- Bird → شامل Parrot و Chicken
- Reptile → شامل Turtle و Crocodile
- Fish → شامل Salmon و Tuna

هر نوع خاص (مثلاً Dog) ساختار خودش را دارد:

```
1 | typedef struct Dog {  
2 |     void (*sound)(char* name);  
3 | } Dog;
```

در اینجا sound یک Function Pointer است که برای شبیه‌سازی رفتار خاص هر حیوان استفاده می‌شود — مشابه متد(تابع)های کلاس(شیء)ها در زبان‌هایی مانند Java یا ++C.

آرایه‌ی اکوسیستم

برای نگهداری لیست حیوان‌ها از یک آرایه‌ی سراسری استفاده شده است:

```
1 | Animal* ecosystem[MAX_ANIMALS];  
2 | int count = 0;
```

- ecosystem لیستی از اشاره‌گر به حیوان‌ها است.
- count تعداد حیوان‌های موجود را مشخص می‌کند.
- تمام کامندها و عملیات روی این آرایه انجام می‌شوند.
- برای دیدن این متغیرها به خط 60 ام کد تمپلیت مراجعه کنید یا سرچ کنید Global Ecosystem

توابع کمکی آماده در کد

در قالب اولیه کد، دو تابع کمکی از پیش تعریف شده‌اند که باید از آن‌ها در بخش‌های مناسب استفاده کنید:

1. void toLowerCase(char* str)

تمام حروف رشته‌ی ورودی را به حروف کوچک (lowercase) تبدیل می‌کند. از این تابع باید هنگام پردازش ورودی کاربر (مانند نوع حیوان یا دستورها) استفاده کنید تا مقایسه‌ی رشته‌ها بدون حساسیت به حروف انجام شود.

مثلاً:

```
1 | toLowerCase(type);
2 | if (strcmp(type, "dog") == 0) { ... }
```

2. int parseAnimalCategory(const char* typeStr)

این تابع یک رشته (مثل "mammal" یا "Fish") می‌گیرد و مقدار متناظر `enum` را برمی‌گرداند. اگر رشته نامعتبر باشد، مقدار `-1` بازمی‌گرداند. این تابع داخل فرمان‌هایی که ورودی نوع دسته (category) می‌گیرند — مثل `speak type <category>` یا فیلترهای `list` — استفاده می‌شود.

مثال استفاده (در یک تابع):

```
1 | int category = parseAnimalCategory(type);
2 | if (category == -1) {
3 |     printf("Unknown category: %s\n", type);
4 | }
```

مثلاً به ورودی تابع `Mammal` را می‌دهیم و فیلد `category` درواقع `enum` آن را در خود نگه می‌دارد، که برای مثال در تابع `void handleSpeakByCategoryCommand(char* category)` که جلوتر به آن اشاره می‌کنیم کاربرد دارد.

3. void printInfo(Animal* self)

این تابع توسط شما باید پیاده‌سازی شود. هدف آن چاپ اطلاعات کامل حیوان با فرمت زیر است: `Type:`

```
%-10s | Specific: %-10s | Name: %-20s | Age: %d\n
```

در بخش بعدی، شما باید توابع ناقص را کامل کرده و منطق اجرای دستورات کاربر در تابع `main` را پیاده‌سازی کنید.

دستورات و توابع Handle

در برنامه، کاربر از طریق ترمینال می‌تواند دستورات زیر را اجرا کند و هر دستور متناظر با یک تابع `handle` است که شما باید پیاده‌سازی کنید. دقت کنید که در هر بخشی که نیاز بود لیستی را چاپ کنید به ترتیبی که حیوانات در آرایی `ecosystem` اضافه شده‌اند اضافه کنید.

1. add animal -t <type> -n <name> -a <age>

- اضافه کردن حیوان جدید به اکوسیستم.
- تابع مرتبط: `handleAddCommand(char* args)`
- داخل این تابع باید از `createAnimal` استفاده کنید و حیوان ایجاد شده را به آرایی `ecosystem` اضافه کنید و `count` را افزایش دهید. (کافیست آرگومان‌ها را جدا کنید و به تابع `createAnimal` پاس دهید.)

پس از اضافه شدن پیغام زیر را چاپ کنید:

```
Added <animal name> (<type>) successfully!
```

مثال:

- ورودی:

```
add animal -t dog -n Rex -a 5
```

- خروجی:

```
Added Rex (dog) successfully!
```

خطاها: تو این بخش خطایی تو تست‌کیس‌ها تعریف نمیکنم حالشو ببرید D:

2. list animals

- نمایش همه حیوان‌های موجود.
- تابع مرتبط: `handleListCommand()`
- داخل این تابع باید برای هر حیوان در `ecosystem` تابع `printInfo` را صدا بزنید.

```
list animals
```

در ابتدا پیام زیر را چاپ کنید:

```
Animals in the ecosystem:
```

اما اگر حیوانی وجود نداشت **فقط** پیغام زیر را چاپ کنید:

```
No animal in the ecosystem.
```

3. speak all

- اجرای صدای همه حیوان‌ها.
- تابع مرتبط: `handleSpeakAllCommand()`
- داخل این تابع باید برای هر حیوان در `ecosystem` تابع `speak` را صدا بزنید.

```
speak all
```

اگر حیوانی وجود نداشت پیغام زیر چاپ شود:

```
No animal to speak.
```

4. speak -c <category>

- اجرای صدای همه حیوان‌های یک دسته مشخص (`FISH` , `REPTILE` , `BIRD` , `MAMMAL`).
- تابع مرتبط: `handleSpeakByCategoryCommand(char* category)`
- در این تابع باید از `parseAnimalCategory` استفاده کنید تا رشته‌ی ورودی کاربر به مقدار `enum` تبدیل شود، سپس فقط حیوان‌های متناظر را صدا بزنید.

```
speak -c Fish
```

اگر حیوانی وجود نداشت پیغام زیر چاپ شود:


```
No animal to speak.
```

خطاها:

- اگره کتگوری مورد نظر وجود نداشت پیغام زیر چاپ شود:

```
Unknown animal category: <category name>
```

مثال:

```
input:
speak -c Goorbeh
output:
Unknown animal category: Goorbeh
```

- اگر هیچ حیوانی در آن کتگوری نبود پیغام زیر چاپ شود:

```
No animal found in category: <category name>
```

- در تمام بخش‌ها ترتیب اولویت خطاها همان ترتیبی هست که ذکر شده‌اند، یعنی اگر مثلاً هم کتگوری وجود نداشت و هم هیچ حیوانی نبود، خطای مربوط به نبود حیوان را چاپ کنید.

5. speak -n <name>

- اجرای صدای یک حیوان خاص با نام مشخص.
- تابع مرتبط: `handleSpeakByNameCommand(char* name)`
- باید در آراییه `ecosystem` به دنبال نام مورد نظر بگردید و اگر پیدا شد `speak` آن حیوان را صدا بزنید.

```
speak -n Rex
```

اگر حیوانی وجود نداشت پیغام زیر چاپ شود:

```
No animal to speak.
```

خطا:

- اگر حیوانی با این نام نبود پیغام زیر چاپ شود:

```
No animal with that name.
```

6. change age -n <name> -a <age>

- تغییر سن یک حیوان مشخص.

- تابع مرتبط: `handleChangeAgeCommand(char* args)`

- باید حیوان مورد نظر را پیدا کنید و فیلد `age` آن را تغییر دهید.

```
change age -n Rex -a 10
```

در اینجا سن رکس به 10 سال تغییر پیدا میکند.

اگر حیوانی وجود نداشت پیغام زیر چاپ شود:

```
No animal to change.
```

پس از تغییر سن پیغام زیر را چاپ کنید:

```
Changed <animal name> age to <new age> successfully!
```

مثال:

- ورودی:

```
change age -n Rex -a 10
```

- خروجی:

```
Changed Rex age to 10 successfully!
```

خطا:

- اگر حیوانی با این نام نبود پیغام زیر چاپ شود:

No animal with that name.

- تضمین می‌شود سن همواره مثبت وارد می‌شود و خطای سن کوچکتر مساوی صفر نداریم.
- اگر کلا حیوانی نبود اون پیغام رو چاپ کنین ولی اگر حیوان داشتیم؛ ولی با اون اسم نداشتیم خطایی که نوشتیم رو چاپ کنین.

exit .7

- پایان برنامه و چاپ پیغام زیر:

Khodafez!

نکته: تمام دستورات روی آرایه‌ی سراسری `ecosystem` و شمارنده `count` انجام می‌شوند.
نکته: تضمین می‌شود نام تکراری برای حیوان‌ها در تست‌ها نمی‌آید. (=

خواندن ورودی و فراخوانی Handleها

در `main` یک حلقه‌ی بی‌نهایت داریم که منتظر ورودی کاربر می‌ماند:

```
1 char input[200];
2
3 while (1) {
4     fgets(input, sizeof(input), stdin);
5     input[strcspn(input, "\n")] = '\0'; // حذف کاراکتر newline
6     // پردازش ورودی و صدا زدن توابع مربوطه
7 }
```

نکات مهم:

۱. جدا کردن دستور و آرگومان‌ها می‌توانید از `strtok` برای جدا کردن بخش‌های دستور استفاده کنید.
برای مثال:

```
char* cmd = strtok(input, " "); // خود دستور (add, list, spea
```

```
char* args = strtok(NULL, ""); // باقی آرگومان‌ها
```

۲. تبدیل رشته‌ها به حروف کوچک

◦ قبل از مقایسه‌ی دسته‌ها یا نوع‌ها، از `toLowerCase` استفاده کنید تا حساسیت به حروف را حذف کنید.

۳. مدیریت حافظه و آزادسازی

◦ در انتهای برنامه، کدی نوشته شده که حافظه‌ی تمام حیوان‌ها و زیرساخت‌ها (`specificCategory`) را آزاد می‌کند تا مدیریت درست حافظه انجام شود (هرچند بعد از اتمام برنامه خود سیستم‌عامل تمام حافظه‌های گرفته شده را آزاد می‌کند ولی مهندسی صحیح این است که خود برنامه‌نویس آزاد کند).

۴. در آخر هم امیدوارم این سوال براتون آموزنده باشه و دیدگاه خوبی نسبت به برنامه‌نویسی شیء‌گرایانه بهتون بده و در کل مفید باشه، موفق باشید:

ورودی نمونه 1:

```
list animals
speak all
change age -n Bozmajeh -a 1
add animal -t dog -n Rex -a 3
add animal -t cat -n Whiskers -a 2
list animals
add animal -t tuna -n Blubby -a 2
speak all
exit
```

خروجی نمونه 1:

```
No animal in the ecosystem.
No animal to speak.
No animal to change.
Added Rex (dog) successfully!
Added Whiskers (cat) successfully!
Animals in the ecosystem:
Type: Mammal      | Specific: Dog      | Name: Rex          | Age:
Type: Mammal      | Specific: Cat      | Name: Whiskers     | Age:
```

```
Added Blubby (tuna) successfully!  
Rex: Hop Hop!  
Whiskers: Meow!  
Blubby: Blub!  
Khodafez!
```

ورودی نمونه 2:

```
add animal -t cat -n ChakmehPoosh -a 3  
add animal -t parrot -n Blu -a 6  
add animal -t dog -n ScoobyDoo -a 10  
add animal -t chicken -n BruceLee -a 4  
add animal -t crocodile -n JooJoo -a 5  
add animal -t turtle -n Oogway -a 200  
add animal -t turtle -n KylianMbappe -a 26  
list animals  
speak all  
speak -c Human  
speak -c Fish  
add animal -t salmon -n Nemo -a 1  
speak -c Fish  
speak -c Reptile  
speak -n Dori  
speak -n BruceLee  
change age -n Arad -a 1  
change age -n ChakmehPoosh -a 7  
change age -n ScoobyDoo -a 9  
list animals  
exit
```

خروجی نمونه 2:

```
Added ChakmehPoosh (cat) successfully!  
Added Blu (parrot) successfully!  
Added ScoobyDoo (dog) successfully!  
Added BruceLee (chicken) successfully!  
Added JooJoo (crocodile) successfully!  
Added Oogway (turtle) successfully!  
Added KylianMbappe (turtle) successfully!
```

Animals in the ecosystem:

Type: Mammal	Specific: Cat	Name: ChakmehPoosh	Age:
Type: Bird	Specific: Parrot	Name: Blu	Age:
Type: Mammal	Specific: Dog	Name: ScoobyDoo	Age:
Type: Bird	Specific: Chicken	Name: BruceLee	Age:
Type: Reptile	Specific: Crocodile	Name: JooJoo	Age:
Type: Reptile	Specific: Turtle	Name: Oogway	Age:
Type: Reptile	Specific: Turtle	Name: KylianMbappe	Age:

ChakmehPoosh: Meow!

Blu: Squawk!

ScoobyDoo: Hop Hop!

BruceLee: Ghod Ghod!

JooJoo: Grrr!

Oogway: ...

KylianMbappe: ...

Unknown animal category: Human

No animal found in category: Fish

Added Nemo (salmon) successfully!

Nemo: Splash!

JooJoo: Grrr!

Oogway: ...

KylianMbappe: ...

No animal with that name.

BruceLee: Ghod Ghod!

No animal with that name.

Changed ChakmehPoosh age to 7 successfully!

Changed ScoobyDoo age to 9 successfully!

Animals in the ecosystem:

Type: Mammal	Specific: Cat	Name: ChakmehPoosh	Age:
Type: Bird	Specific: Parrot	Name: Blu	Age:
Type: Mammal	Specific: Dog	Name: ScoobyDoo	Age:
Type: Bird	Specific: Chicken	Name: BruceLee	Age:
Type: Reptile	Specific: Crocodile	Name: JooJoo	Age:
Type: Reptile	Specific: Turtle	Name: Oogway	Age:
Type: Reptile	Specific: Turtle	Name: KylianMbappe	Age:
Type: Fish	Specific: Salmon	Name: Nemo	Age:

Khodafez!

ArenaAllocator (ویژه‌ی علاقه‌مندان)

این سوال ویژه‌ی علاقه‌مندان است و حل آن شامل نمره‌ی تمرین یا نمره‌ی امتیازی نخواهد شد.

قطعاً تا این‌جای کار با روش‌های مدیریت دستی حافظه، مانند استفاده از `malloc` و `calloc` برای گرفتن حافظه و `free` های متناظر برای آزاد کردن آن، آشنا شده‌اید. اما مشکلی که وجود دارد این است که استفاده از این توابع به‌صورت خام بسیار سخت است و ممکن است به‌سادگی به مشکلاتی مانند *use-after-free* و *free-after-free* برخورد کنید. در واقع، سختی این کار از این نشأت می‌گیرد که شما باید حافظه گرفته‌شده را در مکان و زمان درست آزاد کنید؛ که تشخیص این مسئله به‌صورت بنیادی کار سختی است.

راه دیگری که وجود دارد این است که آزاد کردن حافظه را به سیستم‌عامل واگذار کنیم و در کد هیچ `free` ای نداشته باشیم. اما این کار در اکثر حالت‌ها مناسب نیست؛ چرا که معمولاً برنامه‌های ما قرار است ساعت‌ها یا حتی ماه‌ها اجرا شوند و اگر در یک حلقه حافظه گرفته شود، طولی نمی‌کشد که تمام حافظه سیستم مصرف شده و در نهایت برنامه با خطای پر شدن حافظه بسته می‌شود. آیا بهتر نیست که در جاهای مناسبی، بخش‌های مرتبط حافظه را به‌صورت یک‌جا آزاد کنیم؟

ایده **Arena Allocator** دقیقاً به همین شکل است. ایده این است که شما به‌جای این‌که حافظه را مستقیماً از `malloc` بگیرید و درگیر آزاد کردن آن باشید، تعداد مشخص و منطقی Arena ایجاد می‌کنید و آن‌ها را در انتهای `scope` خود آزاد می‌کنید. سپس هرگاه نیاز به گرفتن حافظه داشتید، از نزدیک‌ترین Arena که تمام استفاده‌های این حافظه را پوشش می‌دهد، حافظه را دریافت می‌کنید.

به‌طور مثال:

```
int main() {
    ArenaAllocator root_arena;
    arena_init(&root_arena);
    // ...
    {
        ArenaAllocator scope_1;
        arena_init(&scope_1);
        char* ptr = 0;
        // ...
        {
            ArenaAllocator scope_2;
            arena_init(&scope_2);
```

```

13         // ...
14         {
15             ArenaAllocator scope_3;
16             arena_init(&scope_3);
17             // ...
18             char* local_ptr = arena_allocate(&scope_1, ALLOCATION_
19             // ...
20             ptr = local_ptr; // local_ptr escapes from scope_2 and
21             // ...
22             arena_destroy(&scope_3);
23         }
24         // ...
25         arena_destroy(&scope_2);
26     }
27     // ...
28     use_ptr(ptr); // outer-most usage of memory ptr=local_ptr
29     // ...
30     arena_destroy(&scope_1);
31 }
32 // ...
33 arena_destroy(&root_arena);
34 }

```

همان‌طور که در بالا مشاهده می‌کنید، صرفاً برای scope هایی که لازم داشتیم حافظه آزاد می‌کنیم در نتیجه مدیریت حافظه بسیار ساده‌تر شده است.

حال سوال زیر را با استفاده از ArenaAllocator حل کنید :

سوال

تعداد $1 \leq q \leq 10^5$ دستور که به صورت zero-based شماره‌گذاری شده‌اند.

حال دستور i ام می‌تواند به انواع زیر باشد:

دستور نوع 1 :

1 n s

ورودی داده می‌شود که $|s| = n$, $1 \leq n \leq 10^5$ و s یک رشته `ascii` متشکل از کلمات کوچک و بزرگ انگلیسیست.

حال یک الگوریتم ساده درهم سازی را روی این رشته انجام می‌دهیم :

```
1 unsigned char hash = 0;
2 for (int i = 0; i < n; i++)
3     hash = (hash ^ s[i]) * s[i] % 256;
```

سپس یک آرایه به طول $ceil(n/16)$ به نام $Arr[i]$ می‌سازیم و آنرا به شکل زیر مقدار دهی می‌کنیم :

```
1 for (knt k = 0; k < (n + 15) / 16; k++) {
2     Arr[i][k] = hash;
3     for (knt j = k * 16; j < 16 * (k + 1) && j < n; j++)
4         Arr[i][k] ^= s[j];
5 }
```

• دستور نوع 2 : $a \ b \ 2$ ورودی داده می‌شود که $a, b < i$ و تضمین می‌شود که a و b اندیس هایی از دستورات نوع 1 هستند. حال باید `hash` را به طور مشابه قبل برای تلفیق یکی در میان $Arr[a]$, $Arr[b]$ انجام دهیم :

```
1 unsigned char hash = 0;
2 for (; i1 < n1 && i2 < n2; i1++, i2++) {
3     hash = (hash ^ Arr[a][i1]) * Arr[a][i1] % 256;
4     hash = (hash ^ Arr[b][i2]) * Arr[b][i2] % 256;
5 }
6 for (; i1 < n1; i1++)
7     hash = (hash ^ Arr[a][i1]) * Arr[a][i1] % 256;
8 for (; i2 < n2; i2++)
9     hash = (hash ^ Arr[b][i2]) * Arr[b][i2] % 256;
```

و در نهایت `hash` را به عنوان خروجی گزارش می‌کنید.

جزئیات پیاده سازی

در این تمرین استفاده از کتابخانه های `C` مجاز نیست.

```

extern char *my_alloc(unsigned long len);
extern void my_free(char *ptr, unsigned long len);
extern unsigned read_int();
extern void read_str(char *buf, unsigned long len);
extern void print_result(unsigned char res);

#define FIRST_BLOCK_SIZE (1 << 6)
#define BLOCK_COUNT 20
// |block[i]| == (FIRST_BLOCK_SIZE << i)

typedef struct { // suggested layout
    char *backing_buffer;
    unsigned long buffer_len;
    unsigned long allocated;
} ArenaBlock;
void arena_block_init(ArenaBlock *block) {
    // TODO: setup initial, unallocated block
}
void arena_block_create(ArenaBlock *block, unsigned long len) {
    // TODO: allocate block
}
char *arena_block_allocate(ArenaBlock *block, unsigned long len) {
    // TODO: try allocating memory from this block
    //      return 0 if failed
}
void arena_block_free(ArenaBlock *block, char *buf, unsigned long len)
    // Optional
}
void arena_block_destroy(ArenaBlock *block) {
    // TODO: release allocated block memory
}
typedef struct { // suggested layout
    ArenaBlock blocks[BLOCK_COUNT];
} ArenaAllocator;
void arena_init(ArenaAllocator *arena) {
    // TODO: setup initial, empty arena
}
void arena_destroy(ArenaAllocator *arena) {
    // TODO: release allocated arena memory
}
char *arena_allocate(ArenaAllocator *arena, unsigned long len) {
    // TODO: allocate memory from arena
}

```

```

44     //      you may choose an optimal block and allocate from it
45 }
46 void arena_free(ArenaAllocator *arena, char *buf, unsigned long len) {
47     // Optional
48 }
49
50
51 void my_main() {
52     ArenaAllocator root_arena;
53     arena_init(&root_arena);
54     // TODO : your code
55     arena_destroy(&root_arena);
56 }

```

روش اجرا

پیشنیازی را دانلود کنید و آنرا آنزیمپ کنید.

کامپایل کردن

با استفاده از دستور `gcc libmain.a your_code.c -o exec` برنامه را کامپایل کنید.

همینطور کد منبع `libmain.a` نیز در کنار آن وجود دارد.

اجرا کردن

با استفاده از دستور `./exec` برنامه را اجرا کنید. در خط اول ورودی حداکثر حافظه مجاز را تعیین کنید. در خط دوم بررسی شدن نشت حافظه را به صورت `true`:بررسی شود , `false`: بررسی نشود تعیین کنید. از اینجا به بعد ورودی به کد شما داده میشود.

به عبارت دیگر :

```

$ ./exec
MAX_MEMORY_IN_BYTES
CHECK_FOR_LEAK
q
1 n s

```

```
2 a b
...
```

مثال :

```
4096
true
4
1 1 a
1 1 b
2 0 1
2 1 0
```

```
164
128
```

توضیح تست :

```
hash[0] = 'a'*'a' % 256 = 193  Arr[0] = ['a'^193] = [160]
```

```
hash[1] = 'b'*'b' % 256 = 132  Arr[1] = ['b'^132] = [230]
```

```
hash([160, 230]) = ((160*160 % 256) ^ 230)*230 % 256 = 164
```

```
hash([230, 160]) = ((230*230 % 256) ^ 160)*160 % 256 = 128
```

```
4096
true
4
1 20 aaaaaaaaaaaaaaaaaaaaaa
1 20 bbbbbbbbbbbbbbbbbbbbbb
2 0 1
2 1 0
```

```
16
0
```

توضیح تست :

```
hash[0] = 64  Arr[0] = [64, 64]
```

```
hash[0] = 124  Arr[0] = [124, 124]
```

```
hash([64, 124, 64, 124]) = ((((((64*64%256) ^124)*124%256) ^64)*64%256) ^124)*124%256 = 16
```

```
hash([124, 64, 124, 64]) = ((((((124*124%256) ^64)*64%256) ^124)*124%256) ^64)*64%256 = 0
```

```
33554432
false
4
1 10 GcJuvOwLJw
1 5 DmXGs
1 3 pDc
2 0 1
```

206

```
33554432
false
13
1 27 TPmjjRwIuwroyIoeQplDqKTUjUN
1 14 RdZhYqOHvPigvi
1 15 bvlduioSHRNzqJw
2 2 2
2 0 1
2 2 0
2 0 0
1 20 UULegZpWNPAlSHYUaisu
1 18 cvFXEZiUEmfgnesuYF
1 6 NgYWdz
1 1 U
2 2 4
2 3 0
```

140

58

136

252

25

140

سیستم نمره دهی

- ۶ تست روی حافظه محدودیت ندارند.
- ۳ تست محدودیت $64nq + 2n + 64q$ بایتی روی حافظه دارند.
- ۴ تست محدودیت $4nq + 2n + 64q$ دارند و همینطور مجاز به نشت حافظه نیستید.
- ۷ تست محدودیت $nq/5 + n + 32q$ روی حافظه دارند و همینطور مجاز به نشت حافظه نیستید.

